

Overview

$\mathcal{X} \rightarrow$ input space
 $\mathcal{Y} \rightarrow$ output space
 $\mathcal{H} \rightarrow$ hypothesis class

Goal:
model $h \in \mathcal{H}$ that best approximates $f: \mathcal{X} \rightarrow \mathcal{Y}$

Supervised Learning: Training

Goal: model h that best approximates f

• One way: empirical risk minimization (ERM)

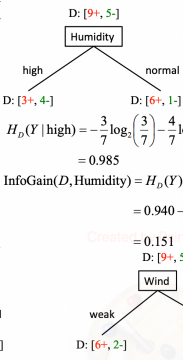
$$\hat{f} = \arg \min_{h \in \mathcal{H}} \frac{1}{n} \sum_{i=1}^n \ell(h(x^{(i)}), y^{(i)})$$

Hypothesis Class Loss function: how far is the prediction from the label?

Model prediction

$$\theta = (X^T X)^{-1} X^T y$$

Humidity	Wind	Play Tennis
High	Weak	No
High	Strong	No
High	Weak	Yes
High	Weak	Yes
Normal	Weak	Yes
Normal	Strong	No
Normal	Strong	No
Normal	Weak	Yes
Normal	Weak	Yes
Normal	Strong	Yes
High	Strong	Yes
Normal	Strong	Yes
High	Strong	Yes
High	Strong	No



InfoGain(D, Humidity) = 0.940 - [7/14 (0.985) + 7/14 (0.592)] = 0.151

GR(D, Humidity) = 0.1518 / 1.0000 = 0.1518

InfoGain(D, Wind) = 0.940 - [8/14 (0.811) + 6/14 (1.0)] = 0.048

GR(D, Wind) = 0.0481 / 0.9852 = 0.0488

Strategy II: 5-fold Cross Validation Example

• Suppose we have 100 instances, and we want to estimate accuracy with 5-fold cross validation

Iteration	train on	test on	correct
1	$s_1 s_2 s_3 s_4 s_5$	s_6	11/20
2	$s_1 s_2 s_3 s_5 s_6$	s_4	17/20
3	$s_1 s_2 s_4 s_5 s_6$	s_3	16/20
4	$s_1 s_3 s_4 s_5 s_6$	s_2	13/20
5	$s_2 s_3 s_4 s_5 s_6$	s_1	16/20

accuracy = 73/100 = 73%

Confusion Matrices: 2-Class Version

predicted class \ actual class	positive	negative
positive	true positives (TP)	false positives (FP)
negative	false negatives (FN)	true negatives (TN)

accuracy = $\frac{TP + TN}{TP + FP + FN + TN}$

error = 1 - accuracy = $\frac{FP + FN}{TP + FP + FN + TN}$

true positive rate (recall) = $\frac{TP}{actual\ pos} = \frac{TP}{TP + FN}$

false positive rate = $\frac{FP}{actual\ neg} = \frac{FP}{TN + FP}$

precision (positive predictive value) = $\frac{TP}{predicted\ pos} = \frac{TP}{TP + FP}$

Category	Parametric Modeling	Non-Parametric Modeling
Definition	Parameters are usually weights; fixed number of parameters	No fixed parameters or weights
Training speed	Fast	Slower to train/predict
Data requirement	Works with less data	Needs more data for accuracy
Flexibility	Limited flexibility	Very flexible
Accuracy with enough data	May underfit	Becomes more accurate
Risk of overfitting	Less prone to overfit	Can overfit
Typical goal	Estimate parameters (θ)	Store data / build structure
Example models	Linear Regression, Logistic Regression, Neural Nets	k-NN, Decision Trees, Kernel Density Estimation

$\sigma(z) = \frac{1}{1 + \exp(-z)} = \frac{\exp(z)}{1 + \exp(z)}$

$P_\theta(y = 1|x) = \sigma(\theta^T x)$

$\hat{\theta} = \arg \max_{\theta} \mathcal{L}(\theta; X)$

Input Space: Feature Types

- **nominal** (including Boolean)
 - no ordering among values (e.g. $color \in \{red, blue, green\}$ (Hertz))
 - **ordinal**
 - values of the feature are totally ordered (e.g. $size \in \{small, medium, large\}$)
 - **numeric (continuous)**
 - $height \in [0, 100]$ inches
 - **hierarchical**
 - possible values are partially ordered in a hierarchy, e.g. $shape$
- MakeSubtree(set of training instances D)
C = DetermineCandidateSplits(D)
If stopping criteria is met
make a leaf node N
determine class label for N
else
make an internal node N
S = FindBestSplit(D, C)
for each group k of S
D_k = subset of training data in group k
kth child of N = MakeSubtree(D_k)
return subtree rooted at N

$H_D(Y) = -\frac{9}{14} \log_2(\frac{9}{14}) - \frac{5}{14} \log_2(\frac{5}{14}) = 0.940$

$H_D(Y | \text{normal}) = -\frac{6}{7} \log_2(\frac{6}{7}) - \frac{1}{7} \log_2(\frac{1}{7}) = 0.592$

$H_D(Y | \text{high}) = -\frac{3}{7} \log_2(\frac{3}{7}) - \frac{4}{7} \log_2(\frac{4}{7}) = 0.985$

InfoGain(D, Humidity) = $H_D(Y) - H_D(Y | \text{Humidity})$

$= 0.940 - [\frac{7}{14} (0.985) + \frac{7}{14} (0.592)] = 0.151$

GR(D, Humidity) = $\frac{0.1518}{1.0000} = 0.1518$

$H_D(Y | \text{weak}) = 0.811$ $H_D(Y | \text{strong}) = 1.0$

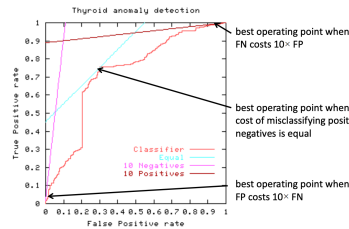
SplitInfo(Humidity) = $-\left(\frac{7}{14} \log_2 \frac{7}{14} + \frac{7}{14} \log_2 \frac{7}{14}\right) = 1.0000$

SplitInfo(Wind) = $-\left(\frac{8}{14} \log_2 \frac{8}{14} + \frac{6}{14} \log_2 \frac{6}{14}\right) = 0.9852$

GR(D, Wind) = $\frac{0.0481}{0.9852} = 0.0488$

- False — in k-NN a large k makes the model simpler (smoother boundary), so it tends to underfit, not overfit.
- Small k (e.g., k = 1): memorizes training points → low bias, high variance, prone to overfitting.
 - Large k: averages over many neighbors → high bias, low variance, decision boundary becomes smooth; as $k \rightarrow \infty$ it predicts the global majority class.

Feature	K-Fold CV	Leave-One-Out CV
# of folds	K (chosen by user)	N (one per sample)
Train/Test size	(K-1)/K train, 1/K test	N-1 train, 1 test
Bias	Slightly higher	Lower
Variance	Lower	Higher
Computation	Moderate	Very high
Typical use	Medium-large datasets	Very small datasets



• logistic regression = sigmoid conditional distribution + MLE

• More precisely:
• Give training data lid from some distribution D,
• Train: $\min_{\theta} \ell(f_{\theta}) = \min_{\theta} -\frac{1}{n} \sum_{i=1}^n \log P_{\theta}(y^{(i)}|x^{(i)})$

• Test: output label probabilities
 $P_{\theta}(y = 1|x) = \sigma(\theta^T x) = \frac{1}{1 + \exp(-\theta^T x)}$

$\theta^{MLE} = \arg \max_{\theta} \prod_{i=1}^n p(x^{(i)}|\theta)$

$\theta^{MAP} = \arg \max_{\theta} \prod_{i=1}^n p(x^{(i)}|\theta) p(\theta)$

• This operation is called softmax.
• Converts a vector into a probability vector (note normalization).
• If one component in the vector a is dominant, softmax(a) is close to the one-hot vector picking out that maximum component

k-Nearest Neighbors: Distances

Discrete features: Hamming distance

$d_H(x^{(i)}, x^{(j)}) = \sum_{a=1}^d \mathbb{1}\{x_a^{(i)} \neq x_a^{(j)}\}$

Continuous features:
• Euclidean distance:
 $d(x^{(i)}, x^{(j)}) = \left(\sum_{a=1}^d (x_a^{(i)} - x_a^{(j)})^2 \right)^{\frac{1}{2}}$

• L1 (Manhattan) dist.:

$d(x^{(i)}, x^{(j)}) = \sum_{a=1}^d |x_a^{(i)} - x_a^{(j)}|$

// Run this subroutine for each numeric feature at each node of DT induction

DetermineCandidateNumericSplits(set of training instances D, feature X_j)

C = {}
// initialize set of candidate splits for feature X_j
let v_j denote the value of X_j for the j^{th} data point
sort the dataset using v_j as the key for each data point
for each pair of adjacent $v_{j_p}, v_{j_{p+1}}$ in the sorted order
if the corresponding class labels are different
add candidate split $X_j \leq (v_{j_p} + v_{j_{p+1}})/2$ to C

Information Theory

Entropy: $H(Y) = -\sum_{y \in \mathcal{Y}} \mathbb{P}(y) \log_2(\mathbb{P}(y))$

$H(Y|X) = \sum_{x \in \mathcal{X}} \mathbb{P}(x) H(Y|X = x)$

Information Gain: $H(Y) - H(Y|X)$

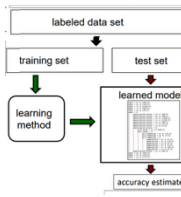
Gain Ratio: $\frac{H(Y) - H(Y|S)}{H(S)}$

Number of distinct binary classification functions with n Boolean features

#functions = 2^{2^n}

Why:

- n Boolean features $\Rightarrow 2^n$ possible input combinations.
- Each input can be labeled 0/1 independently $\Rightarrow 2^{2^n}$ possible labelings (i.e., functions).
- Decision trees can represent any Boolean function, so the number of distinct decision trees as functions equals this.



Strategy I. Stratified Sampling: first stratify instances by class, then randomly select instances from each class proportionally

Strategy II: Cross Validation Tips

- 10-fold cross validation is common, but smaller values folds are often used when learning takes a lot of time
- in leave-one-out cross validation, $k = \#$ instances
- in stratified cross validation, stratified sampling is used when partitioning the data
- CV makes efficient use of the available data for testing
- note that whenever we use multiple training sets, as in CV and random resampling, we are evaluating a learning method (with specific choices) as opposed to an individual learned hypothesis
- You can use CV for tuning as well!

Red curve (Alg 1):

- Stays higher → better performance
- Higher TPR for the same FPR compared to Alg 2
- Covers a larger area under the curve (higher AUC)
- More accurate classifier

Green curve (Alg 2):

- Closer to the diagonal, meaning weaker performance
 - Still better than random guessing but not as good as Alg 1
- Diagonal dashed line:
- Represents random guessing (AUC = 0.5): flipping a coin; no useful predictive power

ROC vs. PR curves

Both

- Allow predictive performance to be assessed at various levels of confidence
- Assume binary classification tasks
- Sometimes summarized by calculating area under the curve

ROC curves

- Invariant to changes in class distribution (ROC curve does not change if the proportion of positive and negative instances in the test set are varied)
- Can identify optimal classification thresholds for tasks with differential misclassification costs

Precision/recall curves

- Show the fraction of predictions that are false positives
- Well suited for tasks with lots of negative instances

- Use ROC when classes are roughly balanced or you care about ranking quality across all thresholds and costs.
- Use PR when positives are rare and you care about the quality of positive alerts (precision) at different recall levels.

• Differentiate (use product rule) and set to 0. Get

$\theta^{k-1} (1 - \theta)^{n-k-1} (k - n\theta) = 0$

• So: ML estimate is

$\hat{\theta} = \frac{k}{n}$

Practice: how about maximum log-likelihood?
 $\log \mathcal{L}(\theta; X) = k \log \theta + (n-k) \log(1-\theta)$
gradient = 0 $\Rightarrow \frac{k}{\theta} - \frac{n-k}{1-\theta} = 0$
 $\Rightarrow \theta = \frac{k}{n}$

• Conditional MLE:

$\log \text{likelihood}(\theta|x^{(i)}, y^{(i)}) = \log P_{\theta}(y^{(i)}|x^{(i)})$

• So

$\min_{\theta} \ell(f_{\theta}) = \min_{\theta} -\frac{1}{n} \sum_{i=1}^n \log P_{\theta}(y^{(i)}|x^{(i)})$

Or

$\min_{\theta} -\frac{1}{n} \sum_{i=1}^n \log \sigma(\theta^T x^{(i)}) - \frac{1}{n} \sum_{y^{(i)}=0} \log(1 - \sigma(\theta^T x^{(i)}))$

$P(H|E) = \frac{P(E|H)P(H)}{\mathcal{P}(E)}$

Posterior Likelihood Prior Evidence (marginal likelihood)

Estimation

Maximum Likelihood:

$\hat{\theta}_{MLE} = \arg \max_{\theta} \mathcal{L}(\theta; X)$

$\mathcal{L}(\theta; X) = \prod_{i=1}^n \mathbb{P}_{\theta}(x_i)$

Conditioned on X:

$\hat{\theta}_{MLE} = \arg \max_{\theta} \mathcal{L}(\theta; Y, X)$

$\mathcal{L}(\theta; Y, X) = \prod_{i=1}^n \mathbb{P}_{\theta}(y_i|x_i)$

Maximum a posteriori Probability:

$\hat{\theta}_{MAP} = \arg \max_{\theta} \prod_{i=1}^n p(x^{(i)}|\theta) p(\theta)$

• Weights only (edges between neurons, no biases):

$d h_1 + h_1 h_2 + h_2 \cdot 1$

• Weights + biases (treating each bias as a connection from a bias node):

$(d+1)h_1 + (h_1+1)h_2 + (h_2+1) \cdot 1$

✓ 1) Bernoulli

Data: $X_1, \dots, X_n \in \{0, 1\}$ Parameter: $p = P(X = 1)$
Probability: $P(X_i = x_i) = p^{x_i}(1-p)^{1-x_i}$
Likelihood: $L(p) = \prod_{i=1}^n p^{x_i}(1-p)^{1-x_i}$
Log-likelihood: $\ell(p) = \sum_{i=1}^n [x_i \ln p + (1-x_i) \ln(1-p)]$
Derivative: $\frac{d\ell}{dp} = \sum_{i=1}^n \frac{x_i - p}{1-p} = 0$
MLE: $\hat{p} = \frac{1}{n} \sum_{i=1}^n x_i$ MAP (Beta a, b): $\hat{p} = \frac{a + \sum x_i - 1}{a + b + n - 2}$ (if $a, b > 1$)

✓ 9) Normal (σ^2 known)

Data: $X_i \in \mathbb{R}$ Parameter: $\mu \in \mathbb{R}$ (known $\sigma^2 > 0$)
Probability: $f(X_i = x_i) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left[-\frac{(x_i - \mu)^2}{2\sigma^2}\right]$
Likelihood: $L(\mu) = \prod f(x_i | \mu)$
Log-likelihood: $\ell(\mu) = -\frac{1}{2\sigma^2} \sum (x_i - \mu)^2 + \text{const}$
Derivative: $\frac{d\ell}{d\mu} = \frac{1}{\sigma^2} \sum (x_i - \mu) = 0$
MLE: $\hat{\mu} = \bar{x} = \frac{1}{n} \sum x_i$ MAP (Normal prior $\mu \sim \mathcal{N}(\mu_0, \tau^2)$): $\hat{\mu} = \frac{(n/\sigma^2)\bar{x} + (1/\tau^2)\mu_0}{n/\sigma^2 + 1/\tau^2}$

✓ 10) Normal (μ, σ^2 both unknown)

Data: $X_i \in \mathbb{R}$ Parameters: $\mu \in \mathbb{R}, \sigma^2 > 0$
Probability: same as above
Likelihood: $L(\mu, \sigma^2) = \prod f(x_i | \mu, \sigma^2)$
Log-likelihood: $\ell(\mu, \sigma^2) = -\frac{n}{2} \ln(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum (x_i - \mu)^2$
Derivatives: $\frac{\partial \ell}{\partial \mu} = \frac{1}{\sigma^2} \sum (x_i - \mu) = 0, \frac{\partial \ell}{\partial \sigma^2} = -\frac{n}{2\sigma^2} + \frac{1}{2(\sigma^2)^2} \sum (x_i - \mu)^2 = 0$
MLE: $\hat{\mu} = \bar{x}, \hat{\sigma}^2 = \frac{1}{n} \sum (x_i - \bar{x})^2$ MAP (Normal-Inverse-Gamma prior): $\hat{\mu} = \mu_n, \hat{\sigma}^2 = \frac{\beta_n}{\alpha_n + 1}$
with $\kappa_n = \kappa_0 + n, \mu_n = \frac{\kappa_0 \mu_0 + n\bar{x}}{\kappa_0 + n}, \alpha_n = \alpha_0 + \frac{n}{2}, \beta_n = \beta_0 + \frac{1}{2} \sum (x_i - \bar{x})^2 + \frac{\kappa_0 n}{2(\kappa_0 + n)} (\bar{x} - \mu_0)^2$

✓ 12) Uniform (0, θ)

Data: $0 < X_i \leq \theta$ Parameter: $\theta > 0$
Probability: $f(X_i = x_i) = \frac{1}{\theta} \mathbf{1}(0 < x_i \leq \theta)$
Likelihood: $L(\theta) = \theta^{-n} \mathbf{1}(\theta \geq \max_i x_i)$
Log-likelihood: $\ell(\theta) = -n \ln \theta$ if feasible, else $-\infty$
Derivative: decreasing in θ on feasible set
MLE: $\hat{\theta} = \max_i x_i$ MAP (Pareto prior $\theta \sim \text{Pareto}(m, \alpha)$): mode = $\max(m, \max_i x_i)$ (for $\alpha + n > 1$)

Linear Regression

Loss: $\ell(f_\theta) = \frac{1}{n} \|\mathbf{X}\theta - \mathbf{y}\|_2^2$
Gradient: $\nabla_\theta = \frac{1}{n} (2\mathbf{X}^T \mathbf{X} \theta - 2\mathbf{X}^T \mathbf{y})$
Solution: $\theta = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$
Ridge Loss: $\ell(f_\theta) = \frac{1}{n} \|\mathbf{X}\theta - \mathbf{y}\|_2^2 + \lambda \|\theta\|_2^2$
Ridge Sol: $\theta = (\mathbf{X}^T \mathbf{X} + \lambda \mathbf{I})^{-1} \mathbf{X}^T \mathbf{y}$

Logistic Regression

Sigmoid: $\sigma(z) = \frac{1}{1 + \exp(-z)} = \frac{\exp(z)}{1 + \exp(z)} \in (0, 1)$
Properties: $1 - \sigma(z) = \sigma(-z)$
 $\sigma'(z) = \sigma(z)(1 - \sigma(z))$
Cross Entropy Loss: $-[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$

• Latent variable $\mathbf{z}$ has some multinomial distribution, $\sum_{i=1}^n \phi_i = 1$
 $z^{(i)} \sim \text{Multinomial}(\phi)$

• Then, let us make $\mathbf{x}$ be Gaussian conditioned on $\mathbf{z}$

$$x^{(i)} | (z^{(i)} = j) \sim \mathcal{N}(\mu_j, \Sigma_j)$$

↑ ↑
Mean Covariance Matrix

• Log likelihood:
 $\ell(\phi, \mu, \Sigma) = \sum_{i=1}^n \log \sum_{z^{(i)}=1}^k p(x^{(i)} | z^{(i)}; \mu, \Sigma) p(z^{(i)}; \phi)$

• Single-linkage $d(A, B) = \min_{x_1 \in A, x_2 \in B} d(x_1, x_2)$

• Complete-linkage $d(A, B) = \max_{x_1 \in A, x_2 \in B} d(x_1, x_2)$

• Average-linkage $d(A, B) = \frac{1}{|A||B|} \sum_{x_1 \in A, x_2 \in B} d(x_1, x_2)$

Naive Bayes

Assumes conditional independence of features:

$$P(X_1, \dots, X_k, Y) \propto P(X_1, \dots, X_k | Y) P(Y) = \left(\prod_{k=1}^K P(X_k | Y) \right) P(Y)$$

2. Step Optimization view $\mathbf{v}$ is vector with maximal variance

$$\mathbf{C} = \frac{1}{n} \mathbf{X}^T \mathbf{X} \quad \mathbf{v}_1 = \arg \max_{\|\mathbf{v}\|=1} \mathbf{v}^T \mathbf{C} \mathbf{v}$$

that is constraint maximization $\max \max_{\|\mathbf{v}\|=1} \mathbf{v}^T \mathbf{C} \mathbf{v}$ where $\mathbf{v}_1 = \mathbf{v}_2 = 1$
theory is to use Lagrange multipliers to solve

$$L(\mathbf{v}, \lambda) = \mathbf{v}^T \mathbf{C} \mathbf{v} - \lambda (\mathbf{v}^T \mathbf{v} - 1)$$

$$\frac{\partial L}{\partial \mathbf{v}} = 2\mathbf{C} \mathbf{v} - 2\lambda \mathbf{v} = 0 \Rightarrow \mathbf{C} \mathbf{v} = \lambda \mathbf{v}$$

So eigenvectors are the solution of equation $\mathbf{C} \mathbf{v} = \lambda \mathbf{v}$

$$L(\theta) = (y - \mathbf{X}\theta)^T (y - \mathbf{X}\theta) + \lambda \|\theta\|_1$$

L2 Regularization

Effect on GD:
Loss: $J_R(\theta) = \hat{L}(\theta) + \frac{\lambda}{2} \|\theta\|_2^2$
Gradient: $\nabla J_R(\theta) = \nabla \hat{L}(\theta) + \lambda \theta$
GD Update: $\theta_{t+1} = (1 - \nu \lambda) \theta_t - \nu \nabla \hat{L}(\theta_t)$
Effect: decays weights by $(1 - \nu \lambda)$
Effect on Optimal Solution:
 $\theta_R^* \approx (H + \lambda \mathbf{I})^{-1} H \theta^* = Q(\lambda + \lambda \mathbf{I})^{-1} \lambda Q \theta^*$
Effect: shrinks along eigenvectors of H

GMM EM: E-Step

• Let us write down the formulas.
• E-step: fix parameters, compute posterior:

$$w_j^{(i)} = p(z^{(i)} = j | x^{(i)}, \phi, \mu, \Sigma)$$

• These w 's are "soft" assignments of the z terms, i.e. probabilities over the values z could take. Concretely:

$$w_j^{(i)} = p(z^{(i)} = j | x^{(i)}, \phi, \mu, \Sigma) = \frac{p(x^{(i)} | z^{(i)} = j; \mu, \Sigma) p(z^{(i)} = j; \phi)}{\sum_{\ell=1}^k p(x^{(i)} | z^{(i)} = \ell; \mu, \Sigma) p(z^{(i)} = \ell; \phi)}$$

GMM EM: M-Step

• Let's write down the formulas.

• M-step: fix w , update parameters:

$$\phi_j = \frac{1}{n} \sum_{i=1}^n w_j^{(i)} \quad \mu_j = \frac{\sum_{i=1}^n w_j^{(i)} x^{(i)}}{\sum_{i=1}^n w_j^{(i)}} \quad \Sigma_j = \frac{\sum_{i=1}^n w_j^{(i)} (x^{(i)} - \mu_j)(x^{(i)} - \mu_j)^T}{\sum_{i=1}^n w_j^{(i)}}$$

Step 3. First eigenvector $\mathbf{v}_1$
1. Start with random vector $\mathbf{v}^0$
2. Repeat until convergence (when direction stops changing)

$$\mathbf{v}^{(k+1)} = \frac{\mathbf{C} \mathbf{v}^{(k)}}{\|\mathbf{C} \mathbf{v}^{(k)}\|}$$

3. Result is $\mathbf{v}_1$ — normalized (unit) eigenvector|
Largest eigenvalue:

$$\lambda_1 = (\mathbf{v}_1)^T \mathbf{C} \mathbf{v}_1$$

• Typical activation function σ

- threshold $h(z) = \mathbf{1}_{\{z \geq 0\}}$
- ReLU $\text{ReLU}(z) = z \cdot \mathbf{1}(z \geq 0) = \max\{0, z\}$
- sigmoid $\sigma(z) = 1/(1 + \exp(-z))$
- hyperbolic tangent $\tanh(z) = 2\sigma(2z) - 1$

✓ 2) Binomial

Data: $K_1, \dots, K_m$ with known $n_1, \dots, n_m$ Parameter: $p \in (0, 1)$
Probability: $P(K_i = k_i) = \binom{n_i}{k_i} p^{k_i} (1-p)^{n_i-k_i}$
Likelihood: $L(p) = \prod_{i=1}^m \binom{n_i}{k_i} p^{k_i} (1-p)^{n_i-k_i}$
Log-likelihood: $\ell(p) = (\sum k_i) \ln p + (\sum n_i - \sum k_i) \ln(1-p)$
Derivative: $\sum k_i - (n_i - k_i) = 0$
MLE: $\hat{p} = \frac{\sum k_i}{\sum n_i}$ MAP (Beta a, b): $\hat{p} = \frac{a + \sum k_i - 1}{a + b + \sum n_i - 2}$ (if $a, b > 1$)

✓ 4) Geometric (failures until first success)

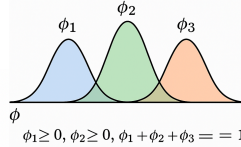
Data: $X_i \in \{0, 1, 2, \dots\}$ Parameter: $p \in (0, 1)$
Probability: $P(X_i = x_i) = (1-p)^{x_i} p$
Likelihood: $L(p) = p^n (1-p)^{\sum x_i}$
Log-likelihood: $\ell(p) = n \ln p + (\sum x_i) \ln(1-p)$
Derivative: $\frac{n}{p} - \frac{\sum x_i}{1-p} = 0$
MLE: $\hat{p} = \frac{n}{n + \sum x_i}$ MAP (Beta a, b): $\hat{p} = \frac{a + n - 1}{a + b + n + \sum x_i - 2}$ (if $a, b > 1$)

✓ 8) Gamma (shape k known, rate β unknown)

Data: $X_i > 0$ Parameter: $\beta > 0$
Probability: $f(X_i = x_i) = \frac{\beta^k}{\Gamma(k)} x_i^{k-1} e^{-\beta x_i}$
Likelihood: $L(\beta) \propto \beta^{nk} e^{-\beta \sum x_i}$
Log-likelihood: $\ell(\beta) = nk \ln \beta - \beta \sum x_i + \text{const}$
Derivative: $\frac{nk}{\beta} - \sum x_i = 0$
MLE: $\hat{\beta} = \frac{nk}{\sum x_i}$ MAP (Gamma(α, β_0) prior on β , rate): $\hat{\beta} = \frac{\alpha + nk - 1}{\beta_0 + \sum x_i}$ (if $\alpha + nk > 1$)

K-means clustering (Lloyd's) algorithm

Input: # clusters k , points $x_1, \dots, x_n$
Step 1: select k cluster centers $c_1, \dots, c_k$
Step 2: for each point $x \in \{x_1, \dots, x_n\}$ determine its nearest cluster center:
 $i_x = \arg \min_i \|x - c_i\|_2$
Step 3: update cluster centers as the centroids:
 $c_i = \frac{\sum_{x: i_x = i} x}{|\{x : i_x = i\}|}$
Repeat step 2 and 3 until the cluster centers no longer change



Step	Formula / Update	Description / Example (for 1-D, 2 components)
E-step	$w_j^{(i)} = p(z^{(i)} = j x^{(i)}, \phi, \mu, \Sigma) = \frac{p(x^{(i)} z^{(i)} = j; \mu_j, \Sigma_j) p(z^{(i)} = j; \phi_j)}{\sum_{\ell=1}^n p(x^{(i)} z^{(i)} = \ell; \mu_\ell, \Sigma_\ell) p(z^{(i)} = \ell; \phi_\ell)}$	For $x^{(1)} = 0, x^{(2)} = 1, x^{(3)} = 5, \phi_1 = \phi_2 = 0.5, \mu_1 = 0, \mu_2 = 5, \sigma_1^2 = \sigma_2^2 = 1$: • $w_1^{(1)} \approx 0.999996, w_1^{(1)} \approx 0.000004$ • $w_1^{(2)} \approx 0.99945, w_2^{(2)} \approx 0.00055$ • $w_1^{(3)} \approx 0.000004, w_2^{(3)} \approx 0.999996$
M-step (update parameters)	$\phi_j = \frac{1}{n} \sum_{i=1}^n w_j^{(i)}$ $\mu_j = \frac{\sum_{i=1}^n w_j^{(i)} x^{(i)}}{\sum_{i=1}^n w_j^{(i)}}$ $\Sigma_j = \frac{\sum_{i=1}^n w_j^{(i)} (x^{(i)} - \mu_j)(x^{(i)} - \mu_j)^T}{\sum_{i=1}^n w_j^{(i)}}$	Soft version of counting estimator: $\phi_1 \approx 2/3, \phi_2 \approx 1/3$ Soft version of mean: $\mu_1 \approx 0.50, \mu_2 \approx 5.00$ Soft version of covariance: $\sigma_1^2 \approx 0.25, \sigma_2^2 \approx 0.009$

Step	Formula	Meaning
Covariance	$C = \frac{1}{n} \mathbf{X}^T \mathbf{X}$	Measure of feature variance
Eigenvector equation	$C \mathbf{v} = \lambda \mathbf{v}$	Find variance directions
Recursive step	$\mathbf{v}^{(k+1)} = \frac{C \mathbf{v}^{(k)}}{\ C \mathbf{v}^{(k)}\ }$	Power iteration
Deflation	$C_k = C - \sum_{i=1}^k \lambda_i \mathbf{v}_i \mathbf{v}_i^T$	Remove previous components
Feature reduction	$Z = \mathbf{X} \mathbf{V}_m$	Project to m principal components

$$\frac{\partial}{\partial x} \tanh(x) = \frac{(e^x + e^{-x})(e^x + e^{-x}) - (e^x - e^{-x})(e^x - e^{-x})}{(e^x + e^{-x})^2} \quad \sigma'(x) = \sigma(x)(1 - \sigma(x))$$
$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = 1 - \frac{(e^x - e^{-x})(e^x - e^{-x})}{(e^x + e^{-x})^2} = 1 - \tanh^2(x)$$

$$h_3 = \sigma\left(\sum_{i=1}^d x_i w_{3i}^{(1)} + b_3\right)$$

✓ 7) Exponential (rate form)

Data: $X_i > 0$ Parameter: $\lambda > 0$
Probability: $f(X_i = x_i) = \lambda e^{-\lambda x_i}$
Likelihood: $L(\lambda) = \lambda^n e^{-\lambda \sum x_i}$
Log-likelihood: $\ell(\lambda) = n \ln \lambda - \lambda \sum x_i$
Derivative: $\frac{n}{\lambda} - \sum x_i = 0$
MLE: $\hat{\lambda} = \frac{n}{\sum x_i} = \frac{1}{\bar{x}}$ MAP (Gamma(α, β), rate): $\hat{\lambda} = \frac{\alpha + n - 1}{\beta + \sum x_i}$ (if $\alpha + n > 1$)

✓ 6) Poisson

Data: $X_i \in \{0, 1, 2, \dots\}$ Parameter: $\lambda > 0$
Probability: $P(X_i = x_i) = e^{-\lambda} \lambda^{x_i} / x_i!$
Likelihood: $L(\lambda) \propto e^{-n\lambda} \lambda^{\sum x_i}$
Log-likelihood: $\ell(\lambda) = -n\lambda + (\sum x_i) \ln \lambda + \text{const}$
Derivative: $-n + \frac{\sum x_i}{\lambda} = 0$
MLE: $\hat{\lambda} = \bar{x} = \frac{1}{n} \sum x_i$ MAP (Gamma(α, β), rate): $\hat{\lambda} = \frac{\alpha + \sum x_i - 1}{\beta + n}$ (if $\alpha + \sum x_i > 1$)

✓ 11) Log-Normal (work on $Y_i = \ln X_i$)

Data: $X_i > 0$ Parameters: $\mu \in \mathbb{R}, \sigma^2 > 0$ for Y
Probability: $f(X_i = x_i) = \frac{1}{x_i \sqrt{2\pi\sigma^2}} \exp\left[-\frac{(\ln x_i - \mu)^2}{2\sigma^2}\right]$
Likelihood: $L(\mu, \sigma^2) = \prod f(x_i | \mu, \sigma^2)$
Log-likelihood: $\ell(\mu, \sigma^2) = -\sum \ln x_i - \frac{n}{2} \ln(2\pi\sigma^2) - \frac{1}{2\sigma^2} \sum (\ln x_i - \mu)^2$
Derivatives: set partials to zero $\rightarrow$ sample stats of $Y = \ln X$
MLE: $\hat{\mu} = \bar{y} = \frac{1}{n} \sum \ln x_i, \hat{\sigma}^2 = \frac{1}{n} \sum (\ln x_i - \bar{y})^2$ MAP (Normal-Inv-Gamma prior on (μ, σ^2) for Y): same closed forms as "Normal (μ, σ^2 unknown)" with $y_i \rightarrow x_i$

Activation Functions

$$\sigma(x) = \frac{e^x}{1 + e^x}$$
$$\text{ReLU}(x) = \max\{0, x\}$$
$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = 2\sigma(2x) - 1$$
$$\frac{d}{dx} \tanh(x) = 1 - \tanh^2(x)$$

EM through the lens of maximal likelihood estimation

• Letting $Q^{(i)} = [Q_1^{(i)}, \dots, Q_k^{(i)}]$ be any distribution over $z^{(i)}$

$$\mathcal{L}(\theta) = \sum_{i=1}^n \log \left(\sum_{j=1}^k Q_j^{(i)} \frac{p_\theta(x^{(i)}, z^{(i)} = j)}{Q_j^{(i)}} \right)$$

• By an application of Jensen's inequality: $f(\mathbb{E}[X]) \geq \mathbb{E}[f(X)]$ for concave f

$$\mathcal{L}(\theta) \geq \sum_{i=1}^n \sum_{j=1}^k Q_j^{(i)} \log \left(\frac{p_\theta(x^{(i)}, z^{(i)} = j)}{Q_j^{(i)}} \right)$$

(For context: prior = ϕ_j ; likelihood = $\mathcal{N}(x^{(i)} | \mu_j, \Sigma_j)$; evidence = $\sum_\ell \phi_\ell \mathcal{N}(x^{(i)} | \mu_\ell, \Sigma_\ell)$; and posterior = $\frac{\text{prior} \cdot \text{likelihood}}{\text{evidence}}$).

Step	Formula	Meaning
Covariance	$C = \frac{1}{n} \mathbf{X}^T \mathbf{X}$	Measure of feature variance
Eigenvector equation	$C \mathbf{v} = \lambda \mathbf{v}$	Find variance directions
Recursive step	$\mathbf{v}^{(k+1)} = \frac{C \mathbf{v}^{(k)}}{\ C \mathbf{v}^{(k)}\ }$	Power iteration
Deflation	$C_k = C - \sum_{i=1}^k \lambda_i \mathbf{v}_i \mathbf{v}_i^T$	Remove previous components
Feature reduction	$Z = \mathbf{X} \mathbf{V}_m$	Project to m principal components

• Ex: 1 hidden layer, 1 output layer: depth 2

